FIELD NOTES

Stripe Field Guide

Stripe fails quietly. A webhook endpoint stops delivering and the payments still succeed, so revenue looks normal while everything that should happen after a payment stops. Every script here is read only.

29 fixes, each one a written note with diagrams and a small Python and Node.js script that finds the problem and repairs it. All 29 have a script in the public repo.

Before you run anything. Every script starts in a dry run: it reports what it would change and writes nothing. Read that output first, then set `DRY_RUN=false` when you are satisfied it has understood your data.

Scripts: github.com/allanninal/stripe-fixes
Guides: allanninal.dev/stripe
All 14 repos: github.com/allanninal

The fixes

1 **3DS handoff breaks and requires_action intents pile up**
Card volume from Europe and India reads lower than your traffic says it should. Nothing fails. There are no declines to look at, no errors in the logs, and no support tickets, because from the customer's side the page simply did nothing. The intents stopped at requires_action and stayed there.
`stripe_requires_action.py`
https://www.allanninal.dev/stripe/abandoned-requires-action-intents/
https://github.com/allanninal/stripe-fixes/tree/main/abandoned-requires-action-intents

2 **automatic_tax is off on every invoice while selling abroad**
Stripe Tax was switched on eighteen months ago, the registrations were filed, and everyone moved on. The invoices going to Germany, France and the UK still carry no VAT, because enabling Tax in the Dashboard did nothing to the subscriptions the API had already created. Nothing errors. The totals are simply wrong, and the liability compounds every month.
`stripe_automatic_tax_off.py`
https://www.allanninal.dev/stripe/automatic-tax-disabled-everywhere/
https://github.com/allanninal/stripe-fixes/tree/main/automatic-tax-disabled-everywhere

3 **Checkout Sessions carry no ID that maps back to your order**

Support is reconciling payments by matching an email address and an amount against the order table by hand. It works, mostly, until two people buy the same thing on the same day. Then a dispute arrives on one of those charges and nobody can say with confidence which order it was, which is a bad position to answer a dispute from.

`stripe_checkout_reconciliation.py`

https://www.allanninal.dev/stripe/checkout-sessions-unreconcilable/

https://github.com/allanninal/stripe-fixes/tree/main/checkout-sessions-unreconcilable

---

4 **a connected account sits with charges_enabled false**

A seller emails support to say their checkout has been broken for two weeks. Nobody on the platform side saw anything: no alert, no failed job, no error in the logs. The platform's own Stripe account is healthy, payments are flowing, the graphs are flat and normal. The account that stopped working is one of four hundred, and the only field that would have told you is one nobody was reading.

`stripe_connect_charges_disabled.py`

https://www.allanninal.dev/stripe/connected-accounts-charges-disabled/

https://github.com/allanninal/stripe-fixes/tree/main/connected-accounts-charges-disabled

---

5 **enabled_events lists event types that are dead or rejected**

Card-expiry reminder emails stopped going out. Nobody can say when, because nothing failed: the handler branch simply never runs, and a branch that never runs produces no logs. Separately, and apparently unrelatedly, an attempt to add one new event type to the endpoint comes back with You do not have access to the event types, naming a type that has been in the list for years.

`stripe_dead_event_types.py`

https://www.allanninal.dev/stripe/dead-or-rejected-enabled-events/

https://github.com/allanninal/stripe-fixes/tree/main/dead-or-rejected-enabled-events

---

6 **disputes are hours from due_by with no evidence attached**

A customer disputed a charge three weeks ago. The notification went to the shared billing inbox, where it sat under invoices. The dispute closed yesterday as lost, the funds went back, the dispute fee did not come back, and the delivery confirmation that would have answered it was in a support ticket the whole time.

`stripe_dispute_deadlines.py`

https://www.allanninal.dev/stripe/dispute-deadline-72h-no-evidence/

https://github.com/allanninal/stripe-fixes/tree/main/dispute-deadline-72h-no-evidence

---

7 **disputes closed as lost were never actually contested**

Somebody finally asks what the dispute win rate is. The Dashboard says most of them are lost, and the conclusion in the room is that disputes are unwinnable and not worth the effort. Nobody in the room can say how many of those losses were ever answered, and until someone can, that conclusion is unsupported.

`stripe_dispute_forfeits.py`

https://www.allanninal.dev/stripe/disputes-lost-without-response/

https://github.com/allanninal/stripe-fixes/tree/main/disputes-lost-without-response

---

8 **draft invoices sit for months and never finalize**

Somebody asks why the March figure is lower than the March that was forecast, and nobody can answer it from the Dashboard, because the money is not missing and it is not late. It is sitting in invoices that were built, itemised, priced correctly, and then never sent: no number, no PDF, no hosted page, no email. Stripe cannot collect on an invoice that has not been finalized, and these have been drafts since March.

`stripe_draft_invoices.py`

https://www.allanninal.dev/stripe/draft-invoices-never-finalized/

https://github.com/allanninal/stripe-fixes/tree/main/draft-invoices-never-finalized

---

9 **dunning ran out of retries and no attempt is scheduled**

A card expired in May. Stripe retried it eight times over two weeks, each attempt failed, and then it stopped, which is exactly what it is supposed to do. Nothing announced the end of that sequence. The invoice is still open, the customer still has access, and the last human to look at the account was the one who set up the subscription.

`stripe_dunning_exhausted.py`

https://www.allanninal.dev/stripe/dunning-retries-exhausted/

https://github.com/allanninal/stripe-fixes/tree/main/dunning-retries-exhausted

---

10 **duplicate customers share an email and split billing**

A customer writes in to say they were charged twice this month. Support finds their subscription, checks it, and it billed once. The second charge is on a different Customer record with the same email address, created the day they resubscribed, and it has its own card, its own subscription and its own renewal date.

`stripe_duplicate_customers.py`

https://www.allanninal.dev/stripe/duplicate-customers-same-email/

https://github.com/allanninal/stripe-fixes/tree/main/duplicate-customers-same-email

---

11 **two endpoints share one URL, so every event is handled twice**

Two order rows for one payment. Two fulfilment emails. A customer credited twice. The handler reads correctly, it passes its tests, and it does not misbehave locally — because locally there is one endpoint, and in production there are two pointing at the same URL.

`stripe_duplicate_endpoints.py`

https://www.allanninal.dev/stripe/duplicate-endpoints-same-url/

https://github.com/allanninal/stripe-fixes/tree/main/duplicate-endpoints-same-url

---

12 **a webhook endpoint is pinned to an ancient api_version**

The signature verifies. The event arrives. And then the object inside it is missing half the fields the current SDK expects, so the deserializer hands back an empty Optional or a null cast, and nobody throws anything. Fetching the same object straight from the API returns it complete, which makes the whole thing look like Stripe sending empty payloads.

`stripe_endpoint_api_version.py`

https://www.allanninal.dev/stripe/endpoint-api-version-pinned-stale/

https://github.com/allanninal/stripe-fixes/tree/main/endpoint-api-version-pinned-stale

---

13 **saved cards are already expired but still attached**

MRR leaks by a percent or two every month and the churn report calls it voluntary. It is not: these customers never chose to leave. Their saved card expired, the renewal failed with expired_card, the dunning emails went to an inbox they do not read, and the account settings page still shows the card as though it works.

`stripe_expired_cards.py`

https://www.allanninal.dev/stripe/expired-saved-cards-attached/

https://github.com/allanninal/stripe-fixes/tree/main/expired-saved-cards-attached

---

14 **payment-creating requests carry no idempotency key**

A customer is charged twice. It happens perhaps once a week, always to someone on a phone, always during a moment when the network was bad, and never once on a developer machine. There is no bug in the checkout code. There is a missing HTTP header, and the events already recorded which requests were sent without it.

`stripe_idempotency_keys.py`

https://www.allanninal.dev/stripe/missing-idempotency-keys-on-payments/

https://github.com/allanninal/stripe-fixes/tree/main/missing-idempotency-keys-on-payments

---

15 **payout.failed is unsubscribed so failures go unseen for days**

Money stopped arriving in the bank account. Nothing alerted, nothing errored, and the balance in Stripe kept climbing. On a Connect platform it is worse: the sellers notice before the platform does, and they notice by not being paid.

`stripe_payout_events.py`

https://www.allanninal.dev/stripe/missing-payout-failed/

https://github.com/allanninal/stripe-fixes/tree/main/missing-payout-failed

---

16 **a connected account has no external account to pay out to**

A seller has been taking payments for five months. Their Stripe balance is a five-figure number and it has only ever gone up. There are no failed payouts to investigate, no errors in any log, and no alert anywhere, for the simple reason that nothing has ever been attempted: the account has no bank account attached, so automatic payouts have nowhere to send the money.

`stripe_missing_external_account.py`

https://www.allanninal.dev/stripe/no-external-account-attached/

https://github.com/allanninal/stripe-fixes/tree/main/no-external-account-attached

---

17 **open invoices are weeks past due_date and nobody chases**

Invoiced customers pay when they are asked twice. The integration asks once: Stripe emails the invoice on finalization and then waits, because that is what collection_method=send_invoice means. Nobody enabled the reminder emails, so an invoice that went out in April is still sitting at open in July, and the customer's subscription never noticed.

`stripe_overdue_invoices.py`

https://www.allanninal.dev/stripe/open-invoices-past-due-date/

https://github.com/allanninal/stripe-fixes/tree/main/open-invoices-past-due-date

---

18 **past_due subscriptions keep their access forever**

Renewals have been failing for months and nobody noticed, because the customers are still logged in and still using the product. The entitlement check in your app asks whether the subscription is canceled. It is not canceled. It is past_due, which the check has never heard of, so the answer is yes, let them in.

`stripe_past_due_subs.py`

https://www.allanninal.dev/stripe/past-due-subscriptions-accumulating/

https://github.com/allanninal/stripe-fixes/tree/main/past-due-subscriptions-accumulating

---

19 **payouts fail with account_closed and nobody is watching**

The ledger says the payout was paid. The recipient says no money arrived. Both are true: the payout reached paid, the bank rejected the credit four days later, Stripe moved it to failed and returned the funds to your balance. Nothing in your system recorded the second half of that story, because nothing was reading the object after it went green.

`stripe_failed_payouts.py`

https://www.allanninal.dev/stripe/payouts-failing-bank-rejection/

https://github.com/allanninal/stripe-fixes/tree/main/payouts-failing-bank-rejection

---

20 **radar blocks payments and nobody reads the block reasons**

Support keeps hearing the same sentence: my card works everywhere else. The charge shows as failed with a message that says nothing, the customer's bank has no record of the attempt at all, and nobody on your side can say why. The payment never left Stripe.

`stripe_radar_blocks.py`

https://www.allanninal.dev/stripe/radar-blocked-payments-ignored/

https://github.com/allanninal/stripe-fixes/tree/main/radar-blocked-payments-ignored

---

21 **refunds sit failed or requires_action and nobody notices**

Support issued the refund, the ticket was closed, and the money left your Stripe balance. Weeks later the same customer opens a dispute for the same transaction, because it never arrived. You now pay the amount twice and the dispute fee on top, and the only record of what went wrong was a status change on an object nobody was watching.

`stripe_refund_health.py`

https://www.allanninal.dev/stripe/refunds-failed-or-stuck/

https://github.com/allanninal/stripe-fixes/tree/main/refunds-failed-or-stuck

---

22 **requirements.past_due has already disabled the payouts**

There is a monitor. It runs daily, it reads every connected account, and it alerts when requirements.currently_due is not empty. It has been green for a month. A seller's payouts stopped eleven days ago and the monitor never said a word about it, because the field that would have told it apart from routine housekeeping is a different array on the same object.

`stripe_requirements_past_due.py`

https://www.allanninal.dev/stripe/requirements-past-due-disables-account/

https://github.com/allanninal/stripe-fixes/tree/main/requirements-past-due-disables-account

---

23 **paymentIntents sit in requires_payment_method for weeks**

The Payments page shows a long tail of incomplete payments that never resolve into anything. Checkout starts and successful payments have drifted apart by a factor nobody can explain, and the gap grows every week. Most of those intents were never going to succeed: they were created before the customer had done anything, and nothing ever went back to them.

`stripe_stale_intents.py`

https://www.allanninal.dev/stripe/stale-requires-payment-method-intents/

https://github.com/allanninal/stripe-fixes/tree/main/stale-requires-payment-method-intents

---

24 **active subscriptions with nothing to charge on renewal**

The subscription says active. The customer has access, the MRR chart counts them, and the renewal date is in the calendar. Then the renewal date arrives and the invoice fails, and it fails again next month, and Stripe never retries any of it — because there is nothing to retry against.

`stripe_sub_payment_method.py`

https://www.allanninal.dev/stripe/subscription-without-payment-method/

https://github.com/allanninal/stripe-fixes/tree/main/subscription-without-payment-method

---

25 **incomplete subscriptions die silently after 23 hours**

Someone filled in the card form, saw a spinner, and closed the tab believing they had subscribed. Stripe has a subscription for them in incomplete. In under a day that record becomes incomplete_expired, the open invoice is voided, and there is nothing left to recover — no charge, no error, no ticket.

`stripe_incomplete_subs.py`

https://www.allanninal.dev/stripe/subscriptions-stuck-incomplete/

https://github.com/allanninal/stripe-fixes/tree/main/subscriptions-stuck-incomplete

---

26 **trials ending in days with no card on file**

Card-free trials are good for signups and they build a queue. Everyone who started a trial on the first of the month reaches its end on the same day, and the ones without a card all fail together. What happens next is one Stripe setting most teams have never opened, and its default is the loudest of the three options and the quietest to you.

`stripe_trial_no_card.py`

https://www.allanninal.dev/stripe/trial-ends-without-payment-method/

https://github.com/allanninal/stripe-fixes/tree/main/trial-ends-without-payment-method

---

27 **undelivered events are aging out of the 30-day window**

The handler is fixed. The endpoint is enabled again. Now someone has to replay three weeks of missed events, and a quiet arithmetic problem is waiting: Stripe keeps events for 30 days, the oldest ones are on day 26, and the backfill script has not been written yet.

`stripe_event_retention.py`

https://www.allanninal.dev/stripe/undelivered-events-nearing-retention/

https://github.com/allanninal/stripe-fixes/tree/main/undelivered-events-nearing-retention

---

28 **a webhook endpoint sits disabled after days of retries**

Orders stopped being fulfilled on a Tuesday. Nothing changed in your code that week, nothing appears in your application logs, and Stripe still shows the payments as succeeded. Nothing is arriving at all — Stripe gave up on the endpoint and stopped trying.

`stripe_webhook_health.py`

https://www.allanninal.dev/stripe/webhook-endpoint-disabled/

https://github.com/allanninal/stripe-fixes/tree/main/webhook-endpoint-disabled

---

29 **an endpoint subscribes to every event and floods the handler**

The webhook route is slow, and it is slow at the worst possible time: the end of the month, when renewals run. It handles four event types. It is being sent everything Stripe generates, and the other ninety-odd types still cost a request, a signature verification, and a database round trip before the handler decides it does not care.

`stripe_wildcard_events.py`

https://www.allanninal.dev/stripe/wildcard-enabled-events/

https://github.com/allanninal/stripe-fixes/tree/main/wildcard-enabled-events

---